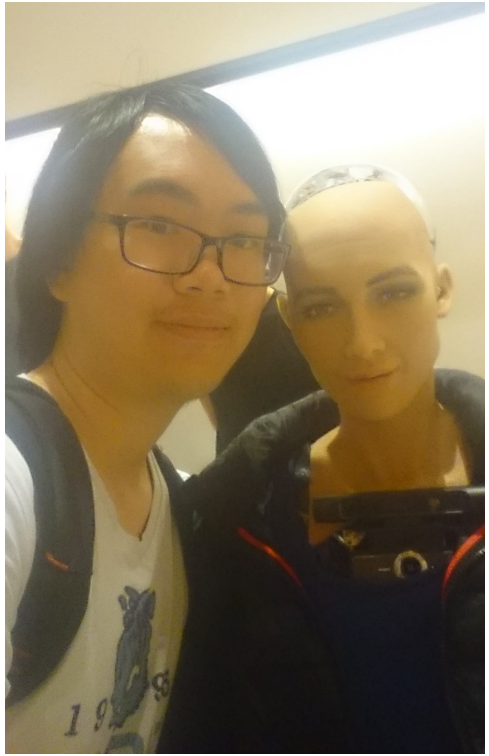
Retrieval Augmented Generation (RAG)

Super AI Engineer Season 6 (Level 1)
27/03/2026

Nutchanon Yongsatianchot
3CHA-AI

About me

- **Name**: Nutchanon Yongsatianchot (Nut)
- B.S. in **Computer Science and Cognitive Science** 2015
  Rensselaer Polytechnic Institute, Albany, NY, USA.
- Ph.D. in **Computer Science** 2021
  Northeastern University, Boston, MA, USA.
- **Research Interest**: Affective Computing, Large Language Models, Human-Computer Interaction
- Been at Thammasat University for almost two years now
- https://www.youtube.com/@3CHA-AI



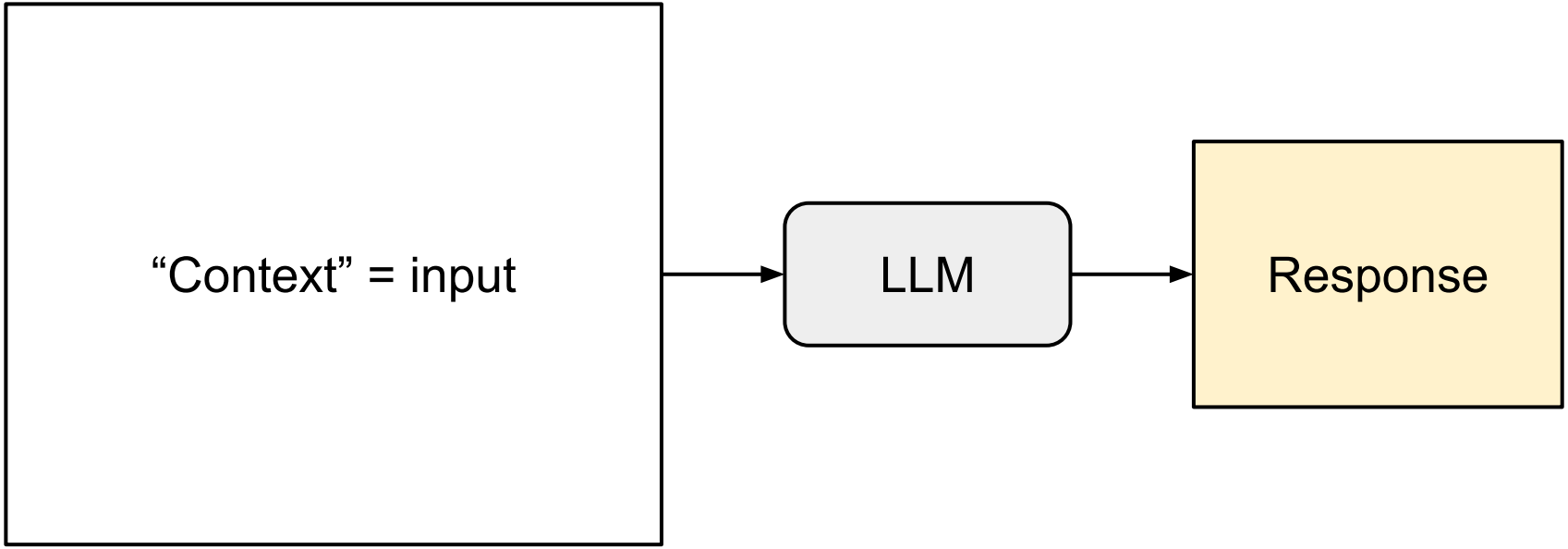
Sophia

RAG Introduction

Some parts adapted from https://www.deeplearning.ai/courses/retrieval-augmented-generation-rag/

- **RAG Introduction**
- Keyword Search
- Semantic Search
- RAG Evaluations
- RAG Improvements
  - Query Rewriting
  - Contextual Retrieval
  - Reranking

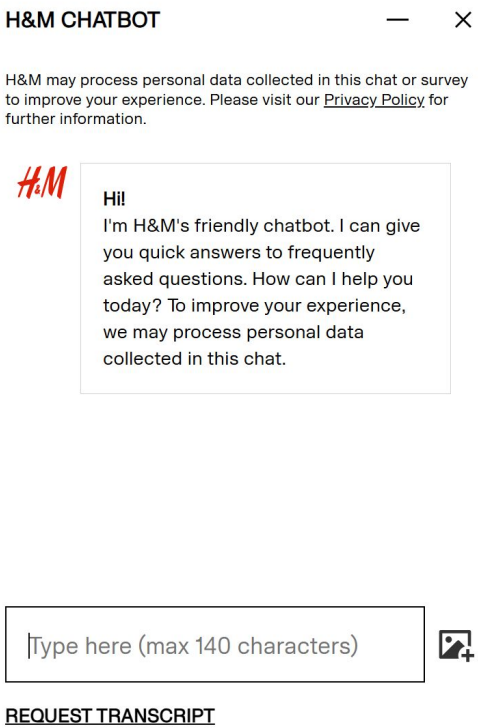
Context



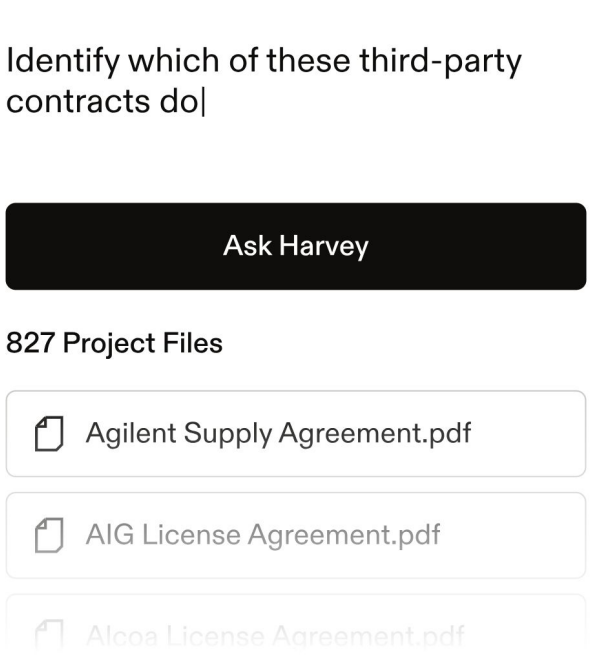
“LLM only knows what in the context and their training data.”

Not in context → Can’t answer

Many applications involve specialized knowledge

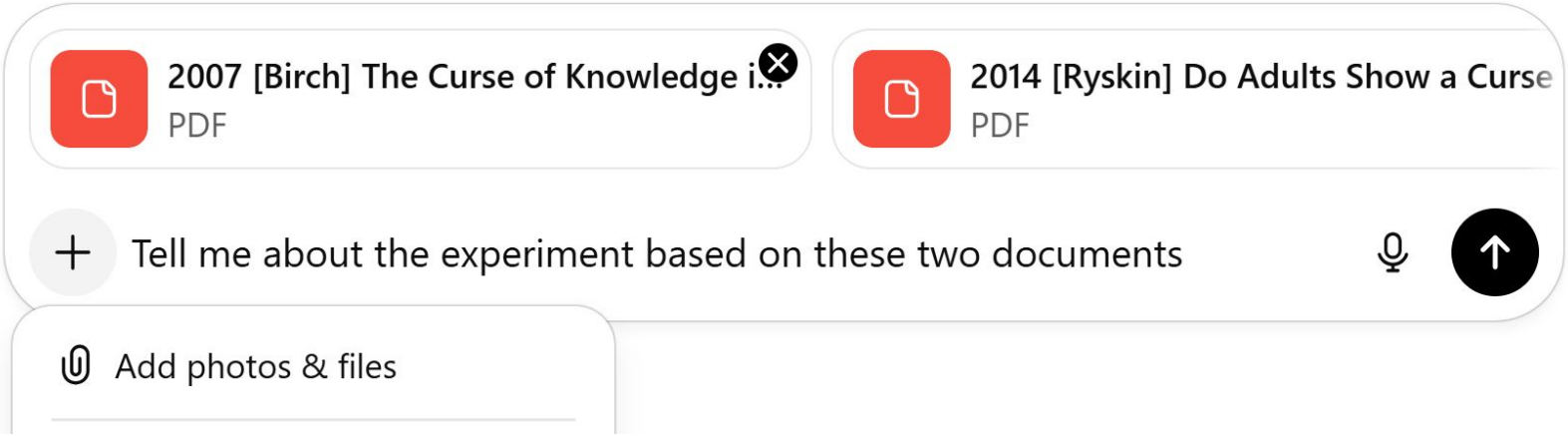


Company chatbot



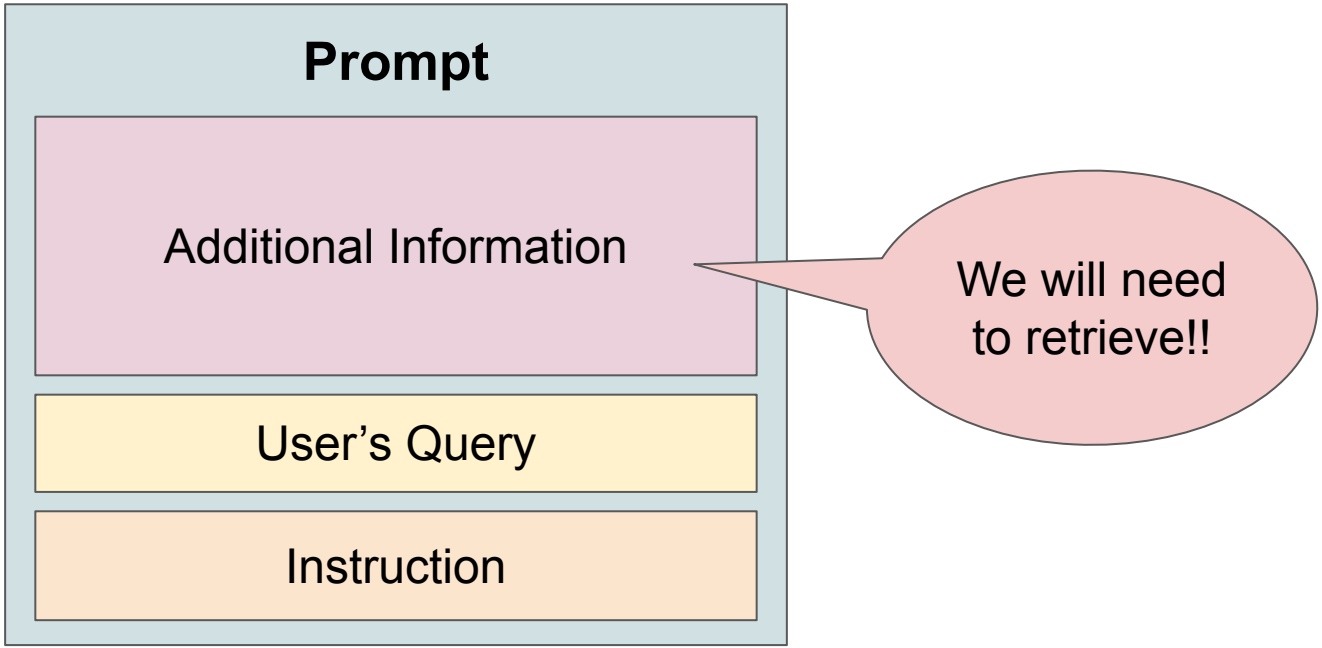
Legal or Medical use cases

Solution: Provide them with needed information



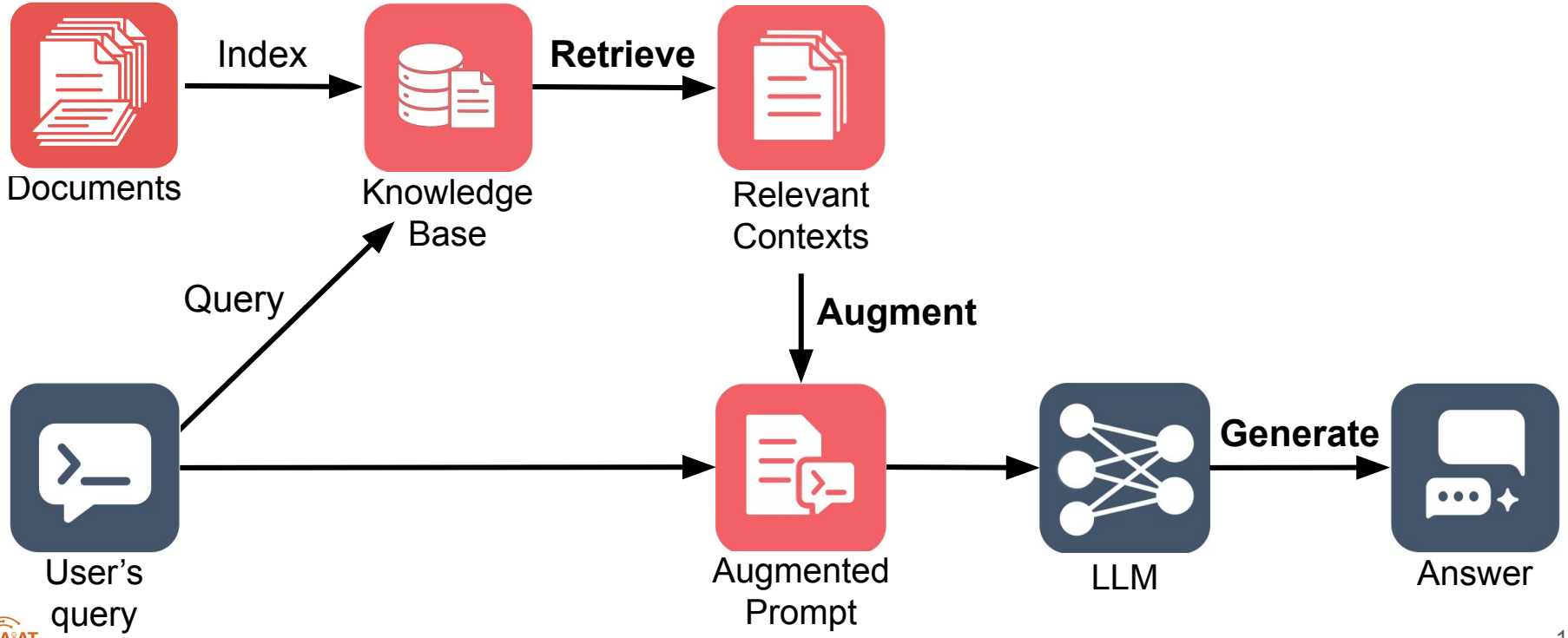
Add relevant documents

Solution: Providing Additional Information



Big question: Can we automate the retrieval?

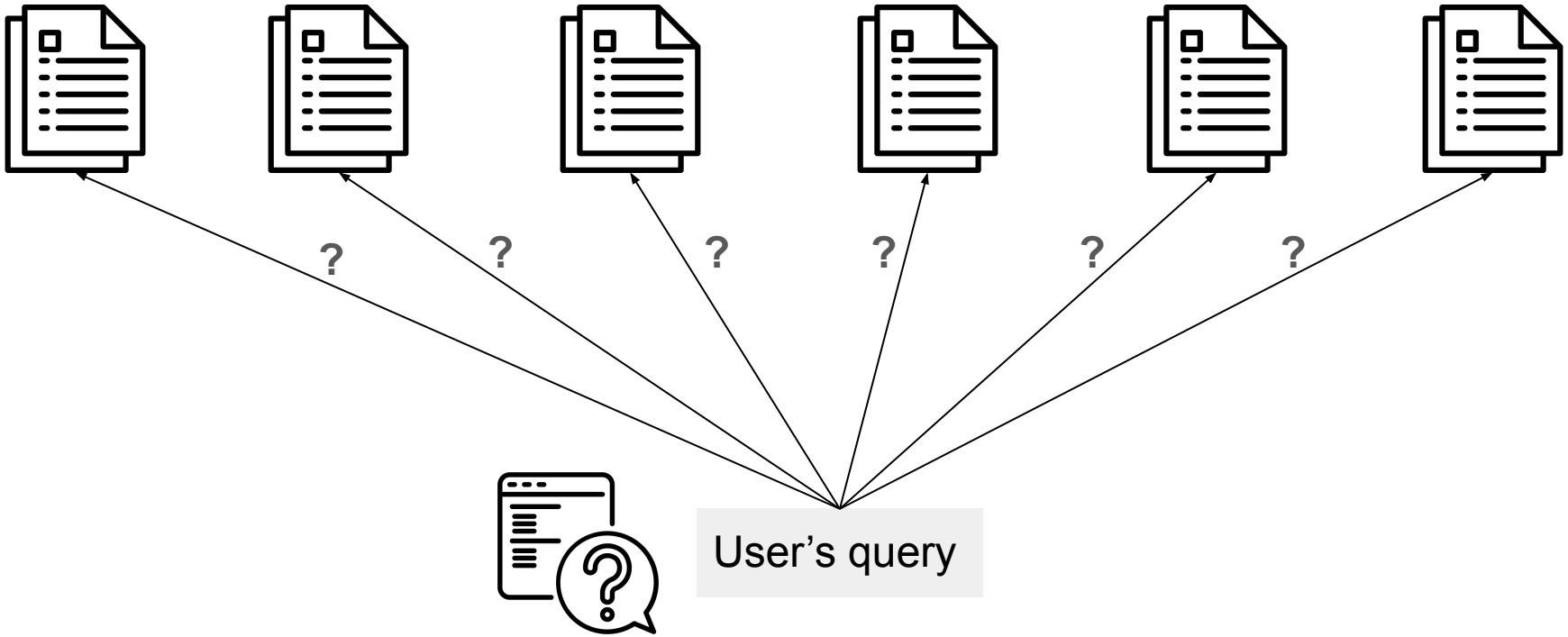
The Big Picture: **R**etrieval **A**ugmented **G**eneration (RAG)



Keyword Search

- RAG Introduction
- **Keyword Search**
- Semantic Search
- RAG Evaluations
- RAG Improvements
  - Query Rewriting
  - Contextual Retrieval
  - Reranking

The problem: Searching relevant documents



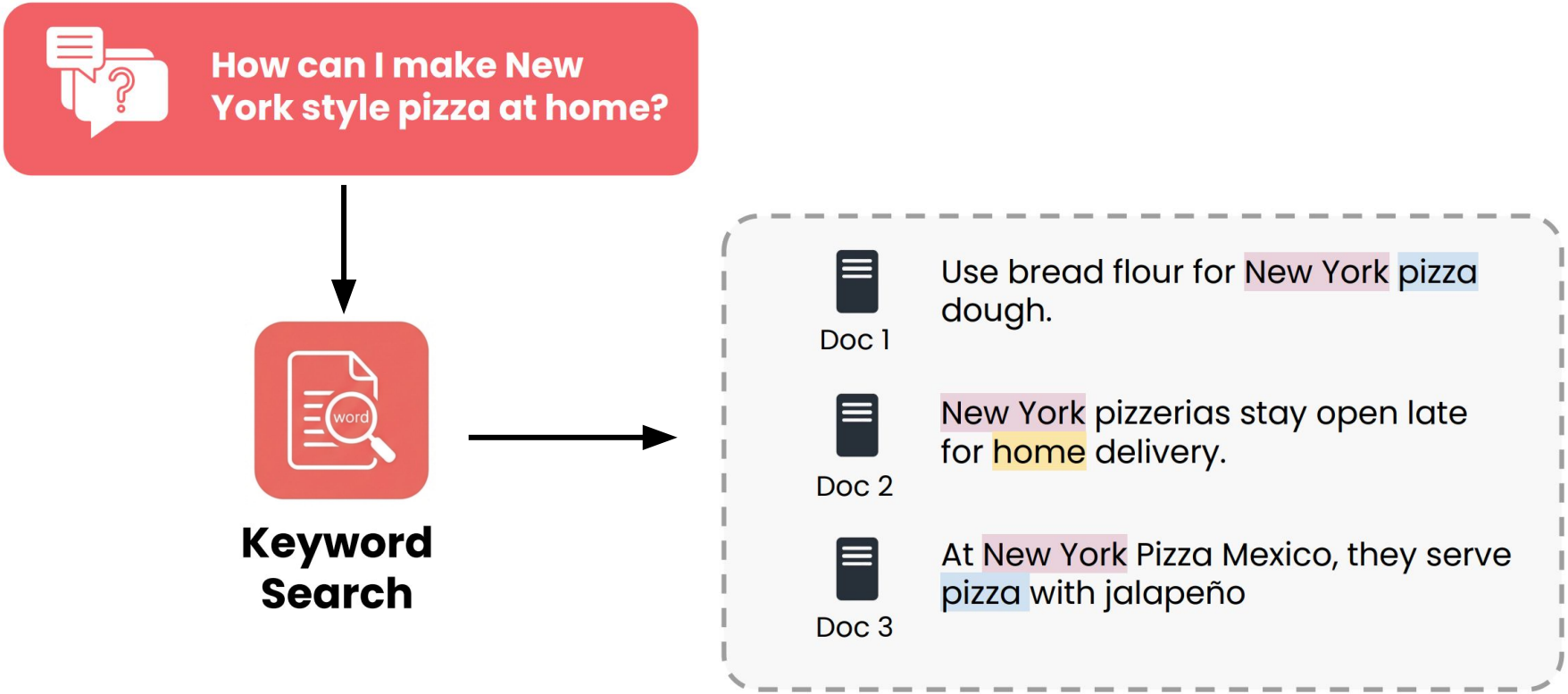
Information Retrieval

Two board ways of retrieving information

- **Sparse**: Keyword Search
  - Bag-of-word Search

- **Dense**: Semantic Search
  - Vector Database

Keyword Search



Term Frequency (TF) Based Scoring

Example Query: pizza oven

Document 1

Homemade pizza in oven is better than frozen pizza

Contains: pizza (2x) oven (1x)

TF Scoring = 3 points

Document 2

Wood-fired oven is a better oven than a stone oven for cooking pizza

Contains: pizza (1x) oven (3x)

TF Scoring = 4 points

Normalized TF Scoring

Longer documents may contain **keywords many times** simply because they are **longer**.

Solution: Normalize by document length

Score = (Number of keyword occurrences) / (Total words in document)

Term Frequency-Inverse Document Frequency (TF-IDF)

Basic TF scoring treats all words equally, whether they're common filler words or rare, meaningful terms.

Solution: Weight terms using “inverse document frequency” (IDF).

Score = TF(word, doc) × log(Total docs / Docs containing word)

1 Count Documents

For each word **making** **pizza**

$$\frac{\text{docs word appears in}}{\text{total docs}}$$

1 Count Documents

Pizza

Appears in 5 out of 100 documents

DF = 5/100 `0.05`

The

Appears in all 100 documents

DF = 100/100 `1.0`

2 Flip to reward rare words

Pizza

High Score

IDF = 1/0.05 `20`

The

Low Score

DF = 1/1.0 `1.0`

3 Apply log

Pizza

Still higher

log(1/20 **)** `1.30`

The

Too common, no weight

log(1 **)** `0`

TF vs. TF-IDF

TF

	Doc 1	Doc 2	Doc 3	Doc 4	Doc 5
making	1			1	1
pizza	1		1		
without	1	1		1	1
a	1	1		1	1
oven	1	1		1	1

→

TF-IDF

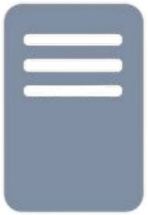
	Doc 1	Doc 2	Doc 3	Doc 4	Doc 5
making	0.4			0.4	0.4
pizza	0.7		0.7		
without	0.2	0.2		0.2	02
a	0.1	0.1		0.1	0.1
oven	0.2	0.2		0.2	0.2

TF-IDF

- **making:** 0.4 (appears in 3/5 docs) moderately common
- **pizza:** 0.7 (appears in 2/5 docs) less common
- **without:** 0.2 (appears in 4/5 docs) very common
- **a:** 0.1 (appears in 4/5 docs) very common
- **oven:** 0.2 (appears in 4/5 docs) very common

	Doc 1	Doc 2	Doc 3	Doc 4	Doc 5
making	0.4			0.4	0.4
pizza	0.7		0.7		
without	0.2	0.2		0.2	02
a	0.1	0.1		0.1	0.1
oven	0.2	0.2		0.2	0.2

“Making pizza without a pizza oven”



High Scoring Document

Pizza	1.3
Oven	1.1

Low Scoring Document

a	0
with	0.1

Documents with **rare keywords** score **higher** than documents with common words

Modern systems use a slightly refined version called BM25

TF-IDF → Refined into → **BM25**

- Keyword rarity
- Term frequency
- Document length

- Document length normalization
- Term Frequency Saturation

Keyword Search Summary

- Retrieve top-k documents based on the keyword scores
- Add these documents to the prompt

Semantic Search

- RAG Introduction
- Keyword Search
- **Semantic Search**
- RAG Evaluations
- RAG Improvements
  - Query Rewriting
  - Contextual Retrieval
  - Reranking

Keyword search does not capture meaning of words

Cannot match synonyms

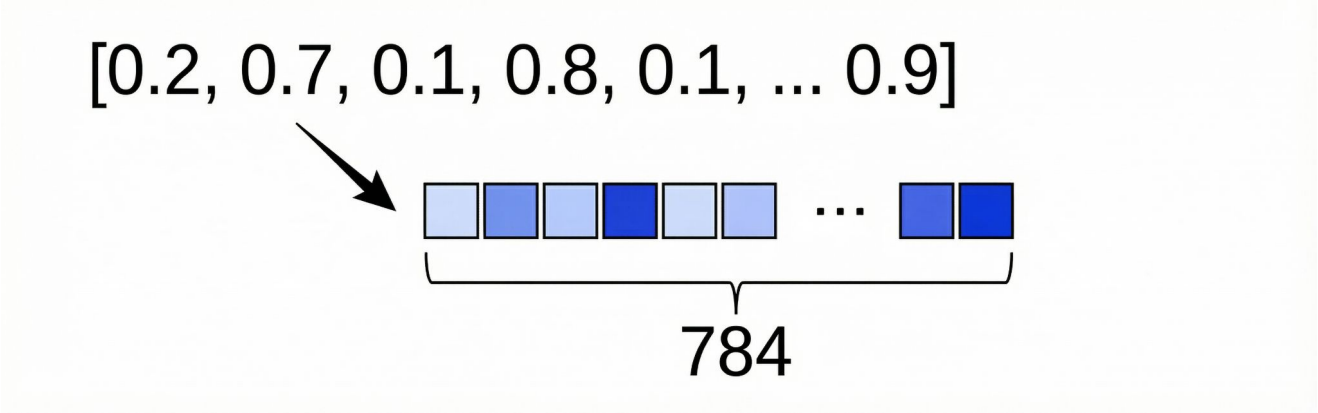
"Happy"

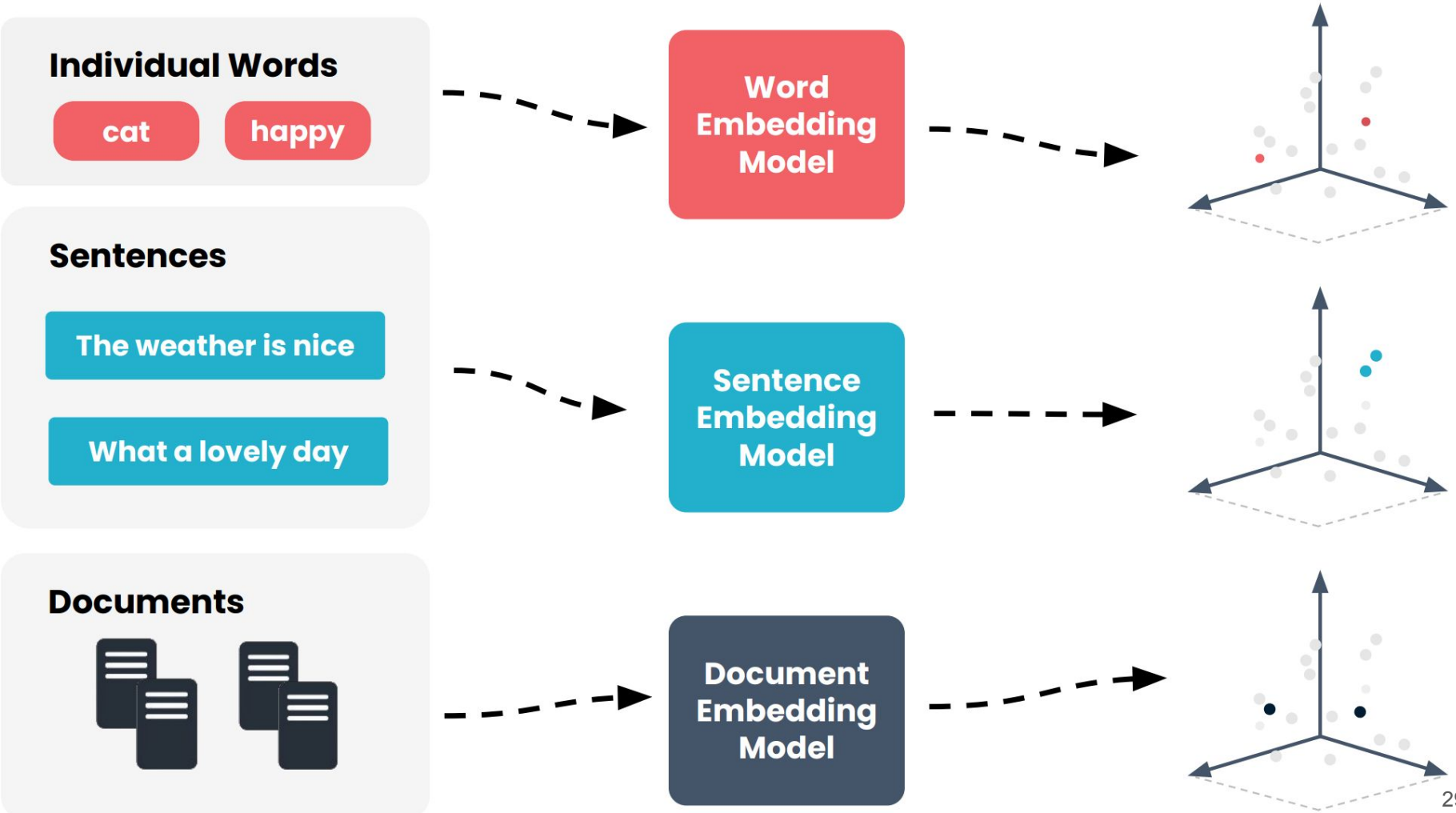
"Joyful"

Incorrectly matches different meanings

Embedding Models

- Embedding models map words or sentences to points in a high-dimensional space, where items with similar meanings end up close together.



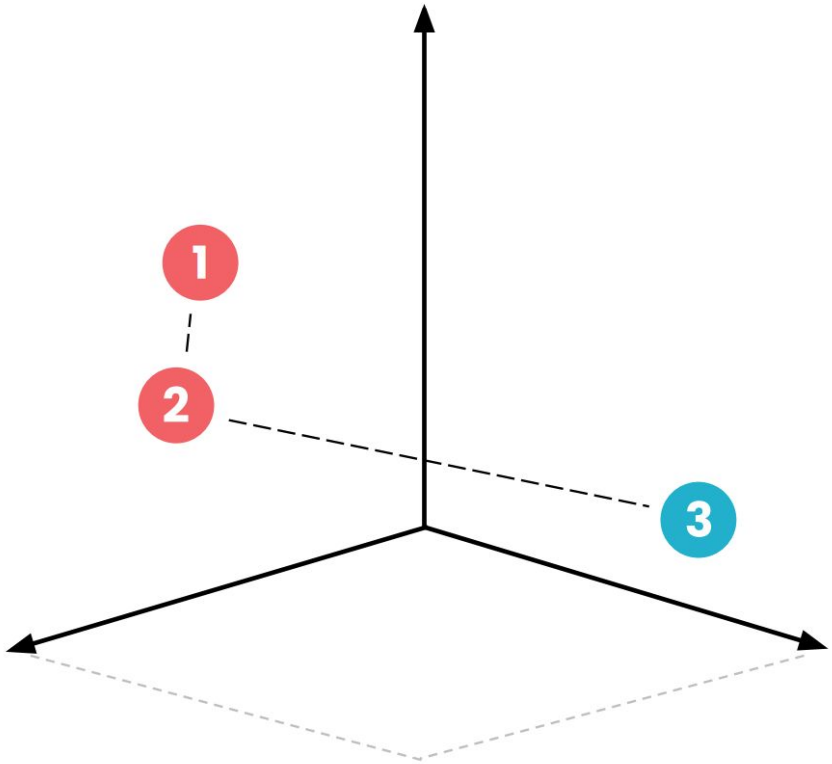

Individual Words
cat
happy
Word Embedding Model
Sentences
The weather is nice
What a lovely day
Sentence Embedding Model
Documents
Document Embedding Model

Sentence Embedding Example

1 He spoke softly in class

2 He whispered quietly during class

3 Her daughter brightened the gloomy day



Measuring Vector Distance

Cosine Similarity

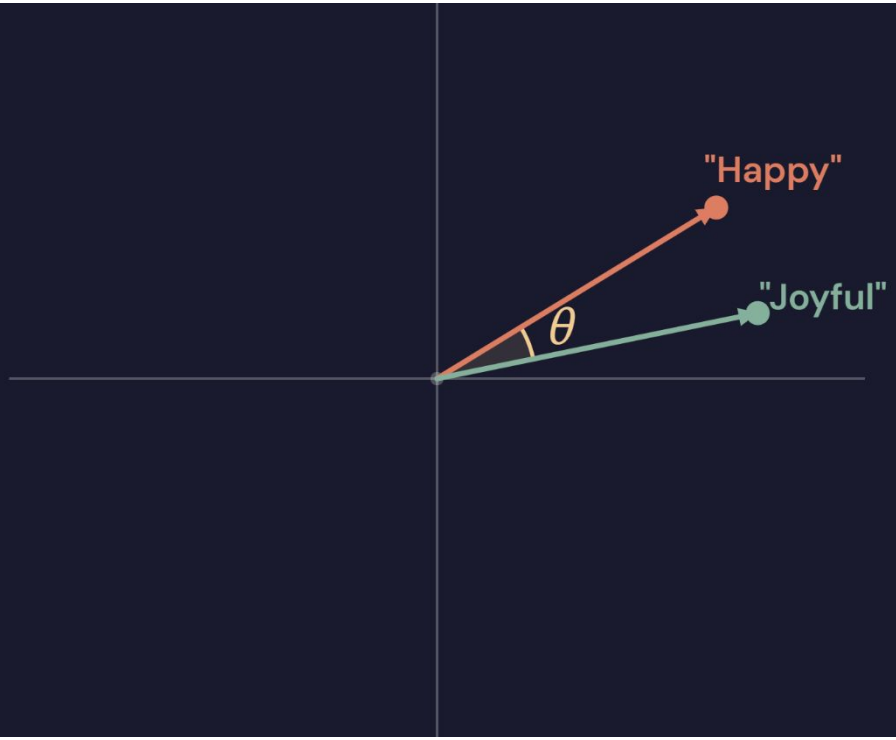
SIMILARITY SCORE

0.94 θ = 20°

-1 0 +1

Cosine similarity measures the angle between two vectors. Same direction (θ = 0°) → similarity = 1. Perpendicular (θ = 90°) → similarity = 0. Opposite directions (θ = 180°) → similarity = -1.

Synonyms | Related | Antonyms | Weak | Opposite



Measuring Vector Distance

Cosine Similarity

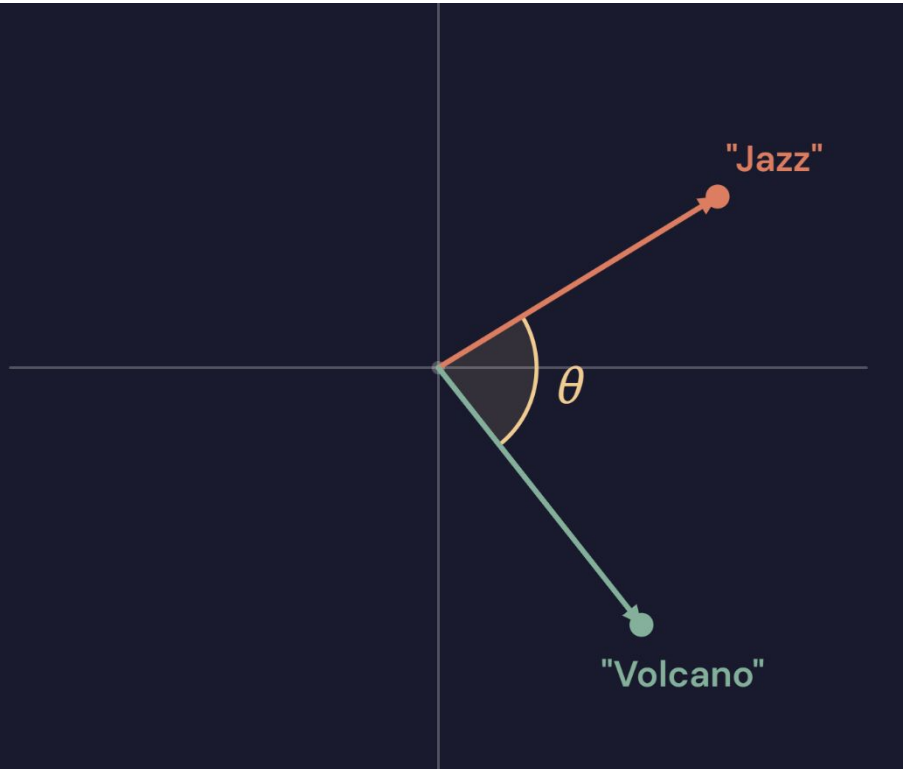
SIMILARITY SCORE

0.12 θ = 83°

-1 0 +1

Cosine similarity measures the angle between two vectors. Same direction (θ = 0°) → similarity = 1. Perpendicular (θ = 90°) → similarity = 0. Opposite directions (θ = 180°) → similarity = -1.

Synonyms | Related | Antonyms | Weak | Opposite



Measuring Vector Distance

Cosine Similarity

SIMILARITY SCORE

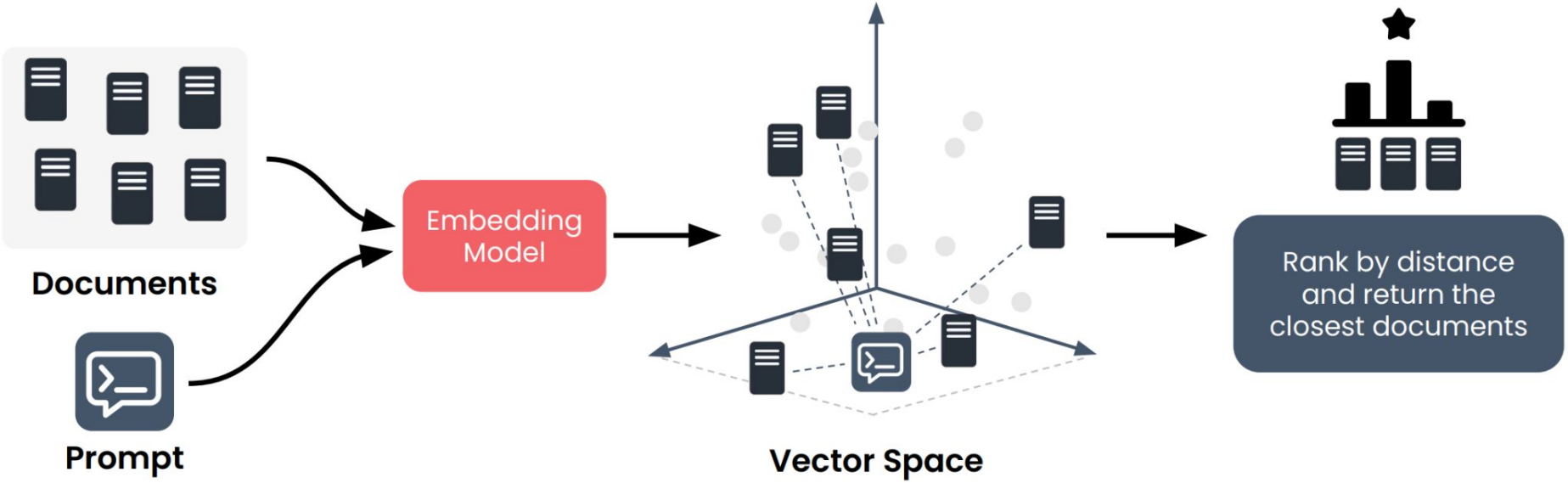
-1.00 θ = 180°

-1 0 +1

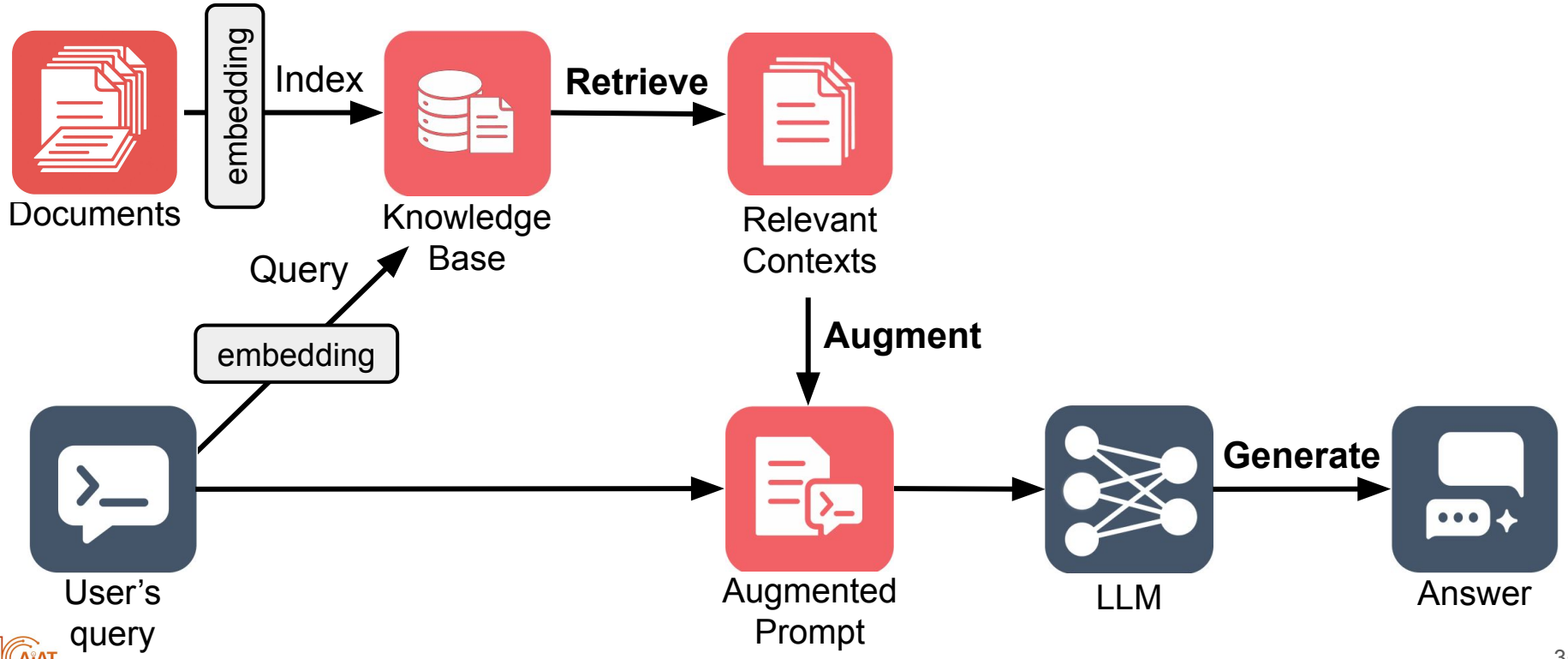
Cosine similarity measures the angle between two vectors. Same direction (θ = 0°) → similarity = 1. Perpendicular (θ = 90°) → similarity = 0. Opposite directions (θ = 180°) → similarity = -1.

Synonyms | Related | Antonyms | Weak | Opposite

Semantic Search Summary



RAG with Semantic Search



Embedding Models

Rank (Bo…	Model	Zero-shot	Memory Us…	Number of P…	Embedding D…	Max Tokens	Mean (T…	Mean (Ta
1	KaLM-Embedding-Gemma3-12B-2511	73%	44884	11.8	3840	32768	**72.32**	**62.51**
2	llama-embed-nemotron-8b	99%	28629	7.5	4096	32768	69.46	61.09
3	gemini-embedding-001	99%			3072	2048	68.37	59.59
4	Qwen3-Embedding-8B	99%	14433	7.6	4096	32768	70.58	61.69
5	Qwen3-Embedding-4B	99%	7671	4.0	2560	32768	69.45	60.86
6	Seed1.6-embedding-1215	89%			2048	32768	70.26	61.34
7	Qwen3-Embedding-0.6B	99%	1136	0.596	1024	32768	64.34	56.01
8	gte-Qwen2-7B-instruct	⚠ NA	29040	7.6	3584	32768	62.51	55.93
9	multilingual-e5-large-instruct	99%	1068	0.560	1024	514	63.22	55.08
10	embeddinggemma-300m	99%	1155	0.308	768	2048	61.15	54.31

https://huggingface.co/spaces/mteb/leaderboard

Vector Database

- **Vector Database**: The database where the generated embedding vectors of documents are stored.
- The key part is the **vector search**
- **Naive Solution (K-Nearest Neighbour Search):**
  1. Compute the similarity across scores between the query embedding and all vectors in the database, using metrics such as cosine similarity
  2. Rank all vectors by their similarity scores
  3. Return k vectors with the highest similarity scores

Technical Problem: What to embed?

- Knowledge base contains 1,000 books
- Each book is vectorized by an embedding model
- **Result**: 1,000 vectors

[0.1,0.4,0.3, …, 0.2, 0.7]

Entire book compressed into a single vector

The problems with this approach

- Compresses entire book meaning into single vector
- Can't sharply represent specific topics, chapters or pages
- Creates "averaged" and “noisy” representation across all content
- Results in poor search relevance
- **Retrieves entire books, quickly filling LLM context window**

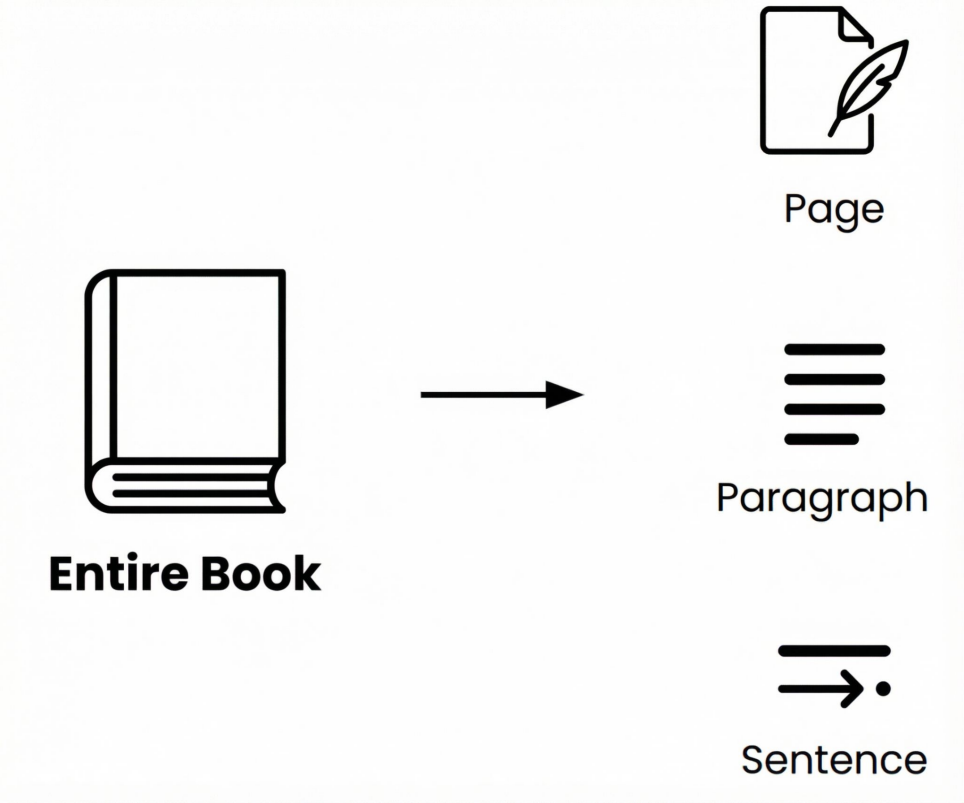
Why Chunk Documents?

Token Limits

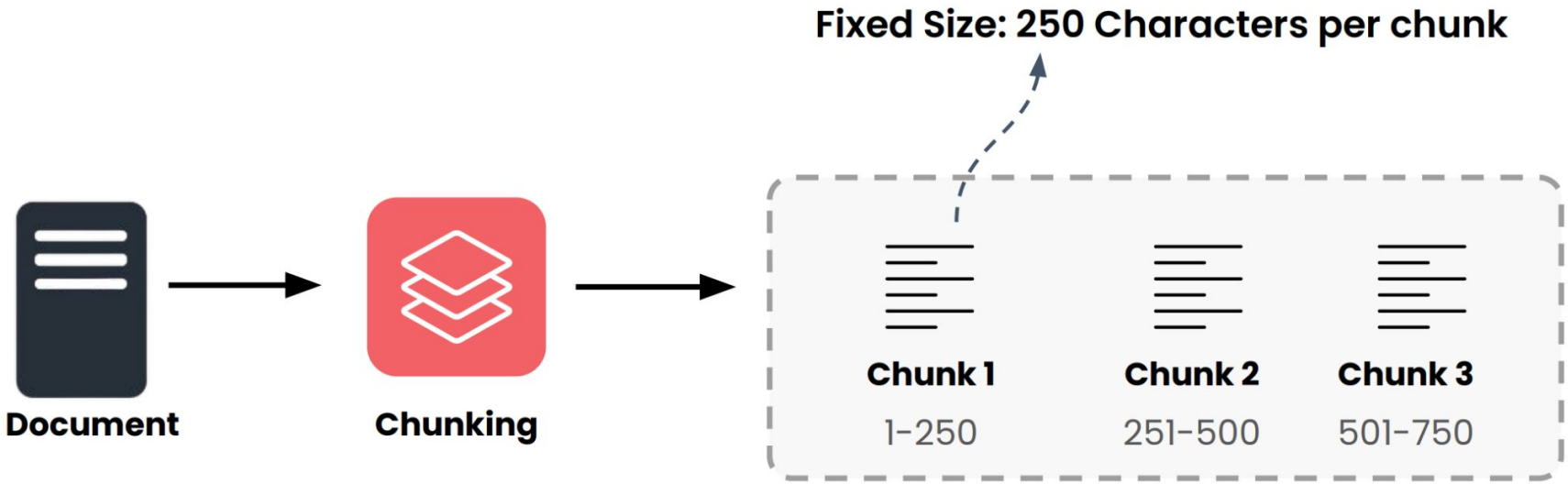
Improved Relevancy

LLM only sent relevant context

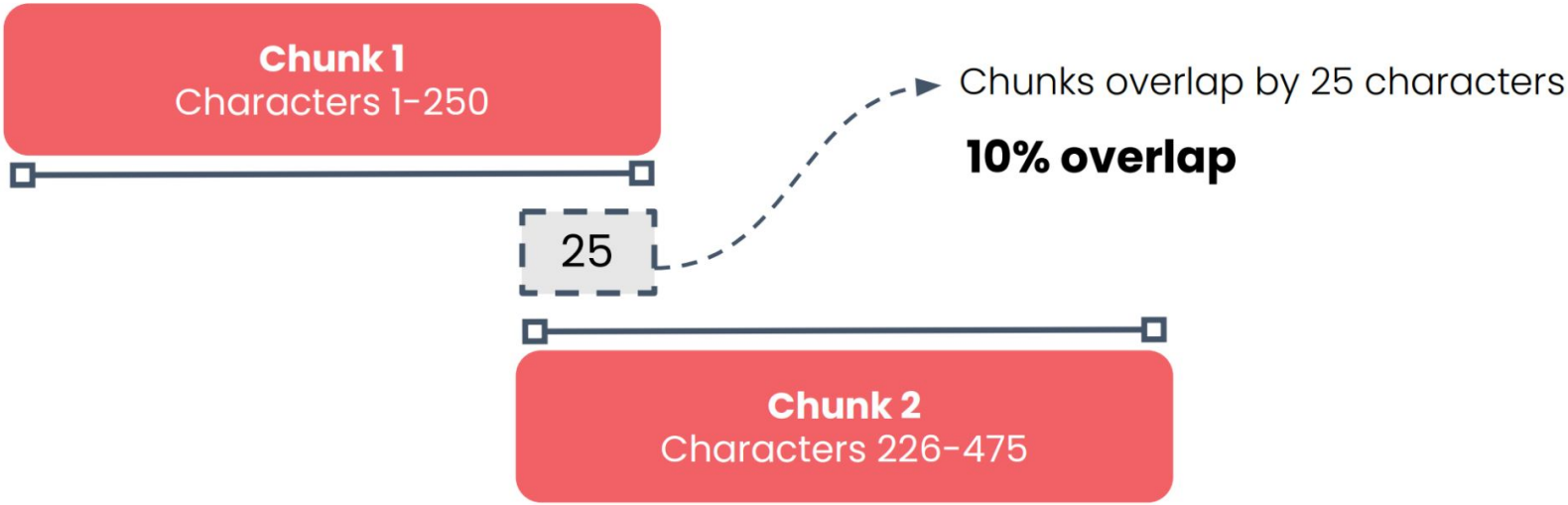
Chunking your content



Fixed Size Chunking



Fixed Size Chunking: Overlapping Chunking



- Minimizes words cut off from context
- Words in the middle of a chunk have context on each side, words on the ends are in two chunks
- Increases relevancy, uses more space

Chunk 3
Characters 451-700

Recursive Chunking

- Recursive chunking divides the input text into smaller chunks **in a hierarchical and iterative manner using a set of separators**.
- If the initial attempt at splitting the text **doesn't produce chunks of the desired size or structure**, the method **recursively calls itself** on the resulting chunks with a different separator or criterion until the desired chunk size or structure is achieved

```python
text = "..." # your text
from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(
    # Set a really small chunk size, just to show.
    chunk_size = 256,
    chunk_overlap  = 20
)
```

Chunking Strategy Demo

https://chunkviz.up.railway.app/

Specialized chunking

- Splitting based on Markdown
- Splitting based on Latex

```python
from langchain.text_splitter import MarkdownTextSplitter
markdown_text = "..."

markdown_splitter = MarkdownTextSplitter(chunk_size=100, chunk_overlap=0)
docs = markdown_splitter.create_documents([markdown_text])
```

Specialized chunking

HTML Documents

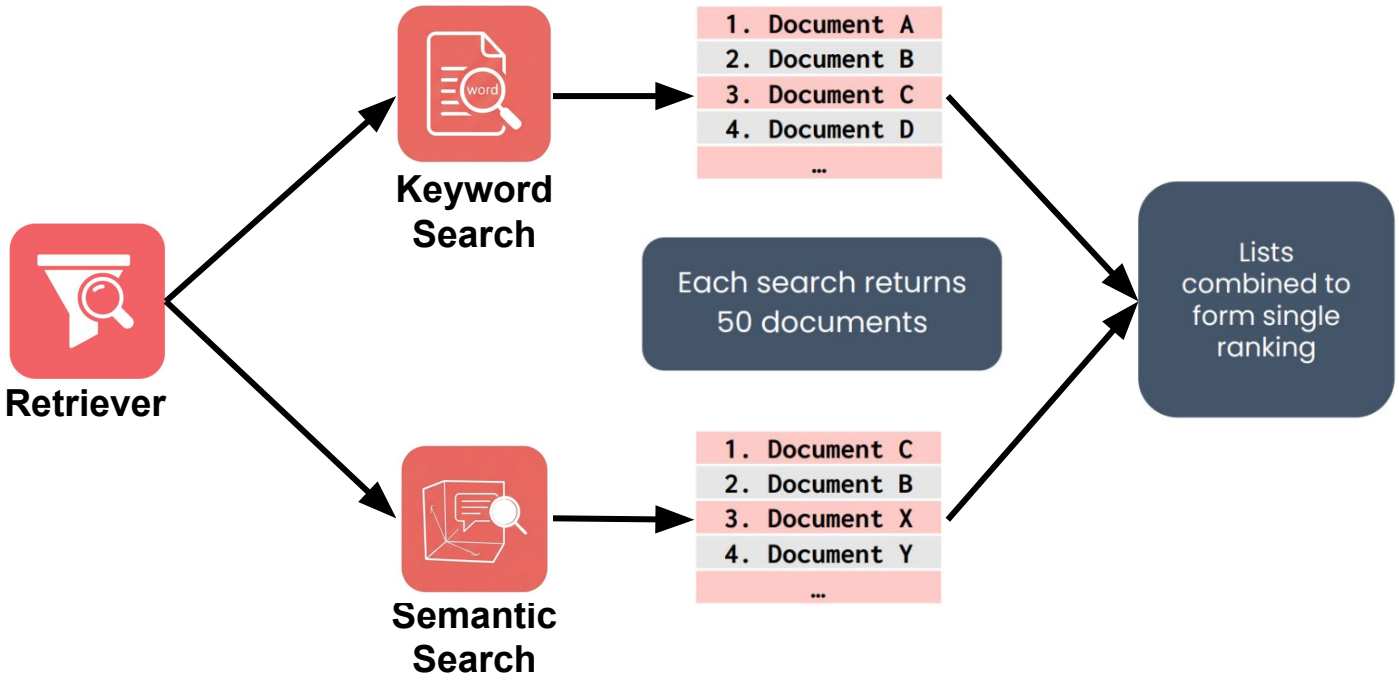
Split paragraph or header characters

Python

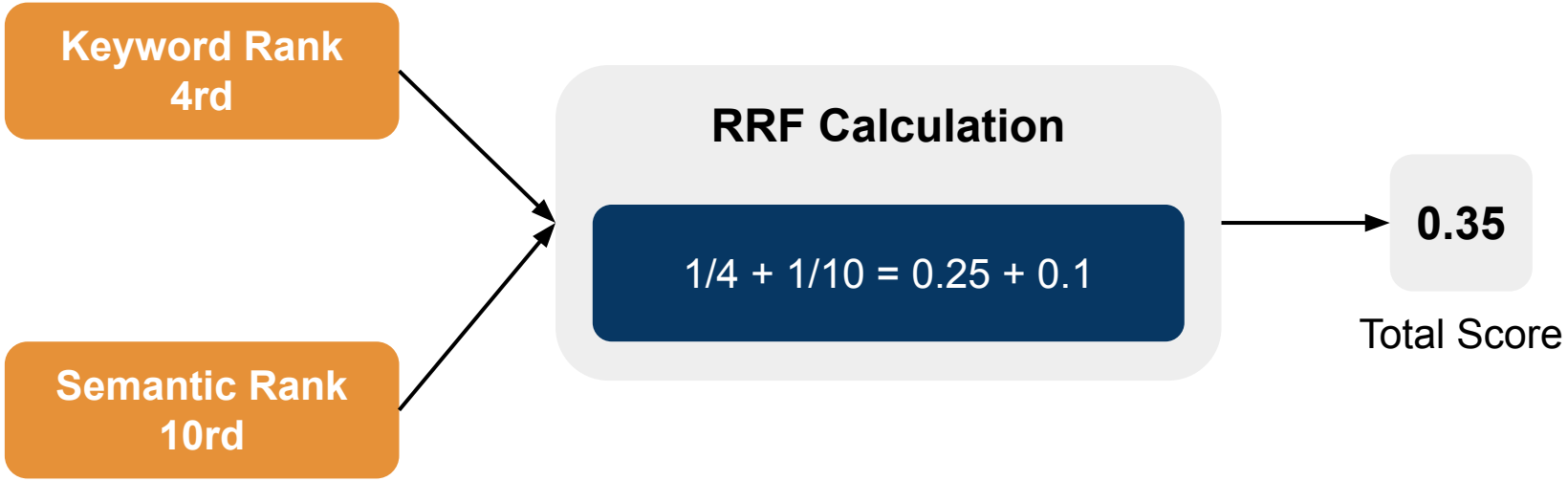
Chunk by function definitions

```
<h1>Document Title</h1>

<p>First
paragraph</p>
<p>Second paragraph</p>

def function_one():
    """Docstring"""
    return result

class
MyClass:
    def
```

Hybrid Search



Reciprocal Rank Fusion



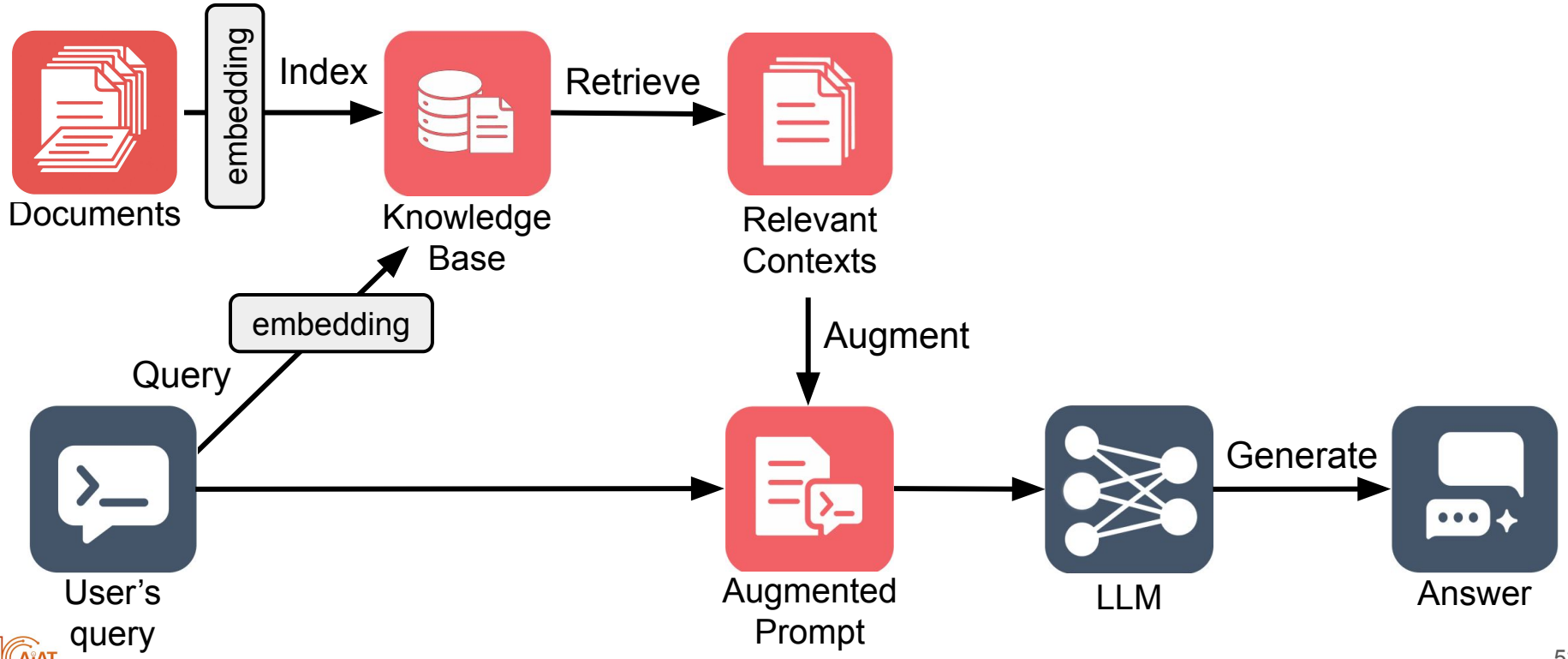
Formula: $$\frac{1}{k + \text{rank in list 1}} + \frac{1}{k + \text{rank in list 2}} + \ldots + \frac{1}{k + \text{rank in list n}}$$

RAG != Semantic Search with Vector Database

RAG Evaluations

- RAG Introduction
- Keyword Search
- Semantic Search
- **RAG Evaluations**
- RAG Improvements
  - Query Rewriting
  - Contextual Retrieval
  - Reranking

RAG Evaluations



RAG Evaluations

RAG systems have three core components:

- A question (Q)
- Retrieved context (C)
- An answer (A)

https://jxnl.co/writing/2025/05/19/there-are-only-6-rag-evals/#tier-2-primary-rag-relationships

Context Relevance (C|Q)

Def: How well do the retrieved chunks address the question information needs? Does your retriever find passages that contain information relevant to answering the user's question.

Bad

Question: "What are the health benefits of meditation?"
Context: "Meditation practices vary widely across different traditions. Mindfulness meditation, which originated in Buddhist practices, focuses on present-moment awareness, while transcendental meditation uses mantras to achieve deeper states of consciousness."
Reasoning: Despite being factually correct about meditation, *this context did not discuss health benefits*.

Good

Question: "What are the health benefits of meditation?"
Context: "Regular meditation has been shown to reduce stress hormones like cortisol. A 2018 study in the Journal of Cognitive Enhancement found meditation improves attention and working memory."
Reasoning: *Strong relevance*. The context directly addresses multiple health benefits with specific details.

Q: What source of information would you need to answer **multiple-choice questions on financial knowledges**?

1. In a purely competitive market, economic losses indicate that:
   A. price is below average total costs.
   B. collusion is occurring in the marketplace.
   C. firms need to expand output to reduce costs.

Finance textbooks

Retrieval Quality Metrics

Common ingredients to most retriever quality metrics:

The Question
The specific question being evaluated

Ground Truth
All documents labeled as relevant or irrelevant

If you want to evaluate your retriever you need to know the correct answers

Recall and Precision

Recall
Measures how many of the relevant documents are returned

Relevant_Retrieved / **Total_Relevant**

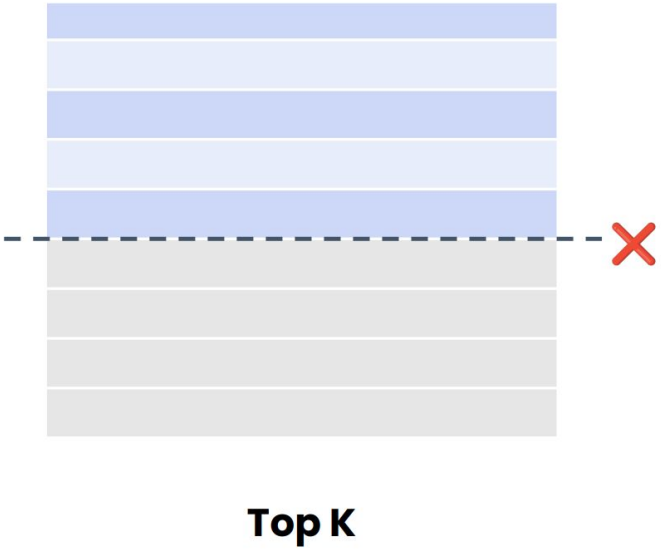
Recall penalizes for leaving out relevant documents

Precision
Measures how many of the returned documents are relevant

Relevant_Retrieved / **Total_Retrieved**

Precision penalizes for returning irrelevant documents

Top k



- Retrieval metrics are influenced by **how many documents the retriever returns**
- Metrics are discussed in terms of **top-k** documents

Example

Rank	Relevant
1	relevant
2	-
3	-
4	relevant
5	-
6	relevant
7	relevant
8	-
9	relevant
10	relevant

Precision @5

2 out of 5

40%

Precision @10

6 out of 10

60%

Recall@10

6 out of 8 total relevant

75%

RAG Evaluations - Edge Cases

Let’s think about edge cases:

- Missing information
- Information not found
- Wrong keyword (e.g., product name)
- Require multiple files/chunks to answer (e.g., comparison)

RAG Improvements

- RAG Introduction
- Keyword Search
- Semantic Search
- RAG Evaluations
- **RAG Improvements**
  - **Query Rewriting**
  - Contextual Retrieval
  - Reranking

RAG Improvements

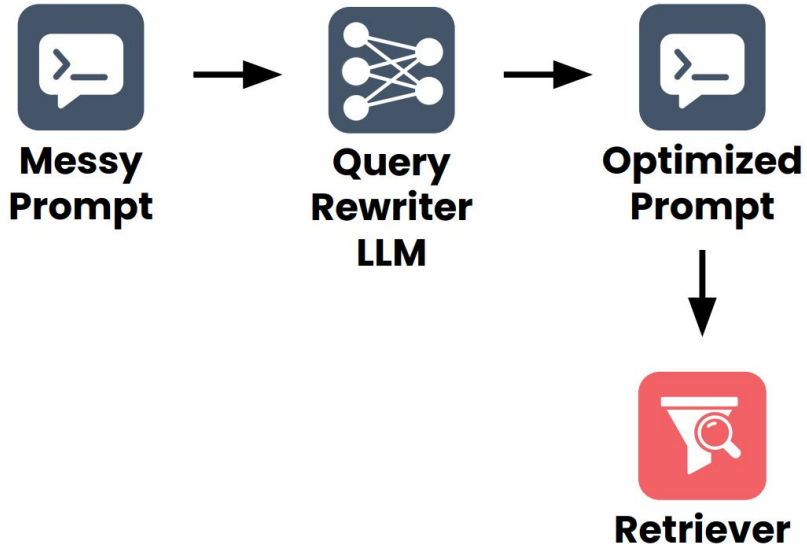
- Improve **Recall**
  - **Query Rewriting** → Make queries better to search
  - **Contextual Retrieval** → Make chunk more informative
- Improve **Precision**
  - **Reranking** → Use a big model to rerank based on relevance

Query Rewriting

- Query Rewriting = Query reformulation, query normalization, and sometimes query expansion.
- **Example**:
  - User: “When was the last time John Doe bought something from us?”
  - AI: “John last bought a Fruity Fedora hat from us two weeks ago, on Jan 3, 2023.”
  - User: “How about Emily Doe?”
  - **Rewriting**: “When was the last time Emily Doe bought something from us?”
- Another common issue is multiple questions in one query → need to expand

Query Rewriting

Use an LLM to **rewrite** the query **before** it's submitted to the retriever.



The following prompt was submitted by a user in order to query a database of medical documents linking symptoms to diagnosis. Rewrite the prompt to optimize it for searching the database by doing the following:

- Clarify ambiguous phrases
- Use medical terminology where applicable
- Add synonyms that increase odds of finding matching documents
- Remove unnecessary or distracting information.

{user prompt}

Patient's Original Prompt

I was out walking my dog, a beautiful black lab named Poppy, when she raced away from me and yanked on her leash hard while I was holding it.

Three days later my shoulder is still numb and my fingers are all pins and needles. What's going on?

LLM Query Rewriter

Optimized Prompt

Experienced a sudden, forceful pull on the shoulder, resulting in persistent shoulder numbness and finger numbness for three days. What are the potential causes or diagnoses, such as neuropathy or nerve impingement?

RAG Improvements

- RAG Introduction
- Keyword Search
- Semantic Search
- RAG Evaluations
- **RAG Improvements**
  - Query Rewriting
  - **Contextual Retrieval**
  - Reranking

Problem

- A chunk may not contain all information to answer the question
- Example:
  - **Q**: "What was the revenue growth for ACME Corp in Q2 2023?"
  - **A relevant chunk might contain the text**: "The company's revenue grew by 3% over the previous quarter."
  - However, this chunk on its own doesn't specify which company it's referring to or the relevant time period, making it difficult to retrieve the right information or use the information effectively.

Contextual Retrieval

- Contextual Retrieval solves the problem by **prepending chunk-specific explanatory context to each chunk** before embedding.

original_chunk = "The company's revenue grew by 3% over the previous quarter."

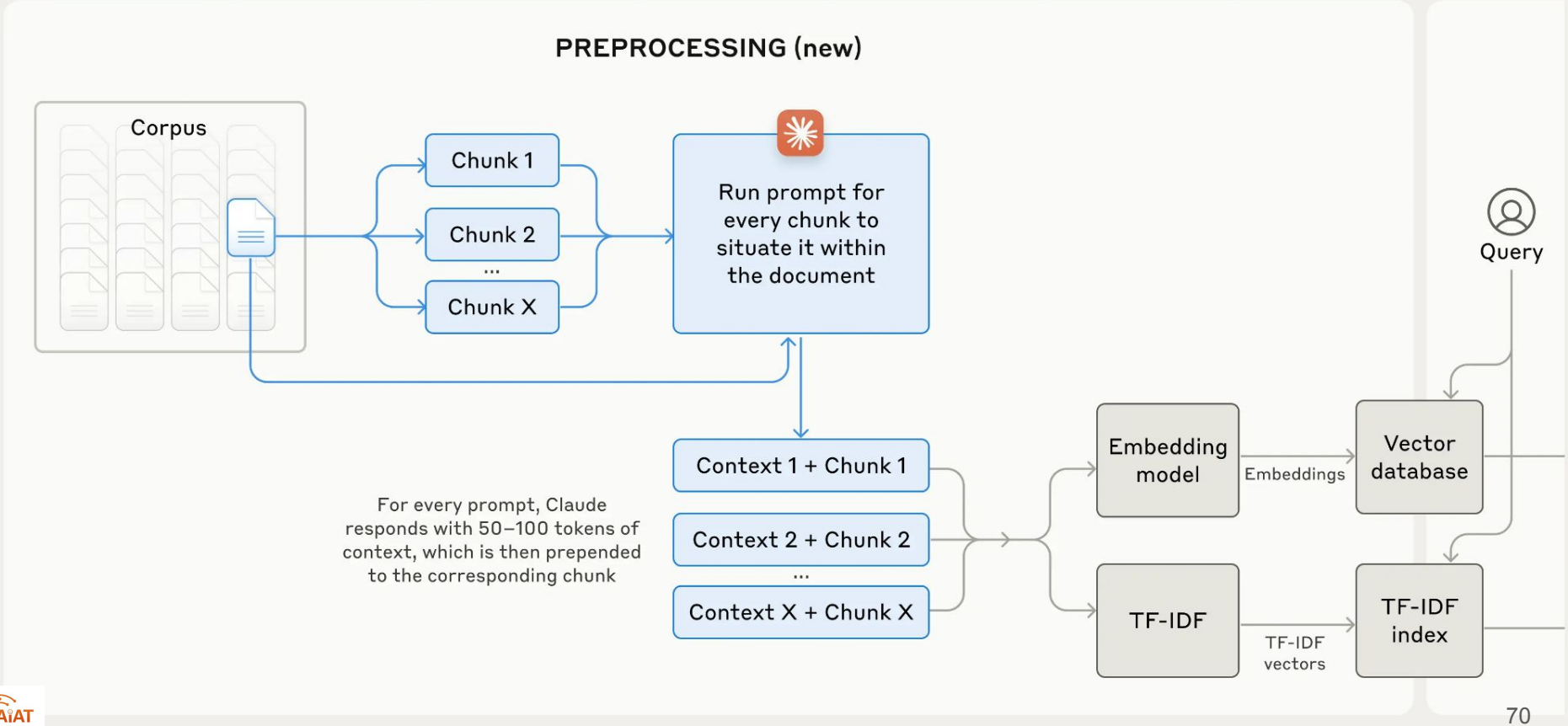
contextualized_chunk = "This chunk is from an SEC filing on ACME corp's performance in Q2 2023; the previous quarter's revenue was $314 million. The company's revenue grew by 3% over the previous quarter."

Implementing Contextual Retrieval

- Use LLM to create contextual Retrieval
- Example Prompt:

```
<document>
{{WHOLE_DOCUMENT}}
</document>
Here is the chunk we want to situate within the whole document
<chunk>
{{CHUNK_CONTENT}}
</chunk>
Please give a short succinct context to situate this chunk within the overall document for the purposes of improving search retrieval of the chunk. Answer only with the succinct context and nothing else.
```

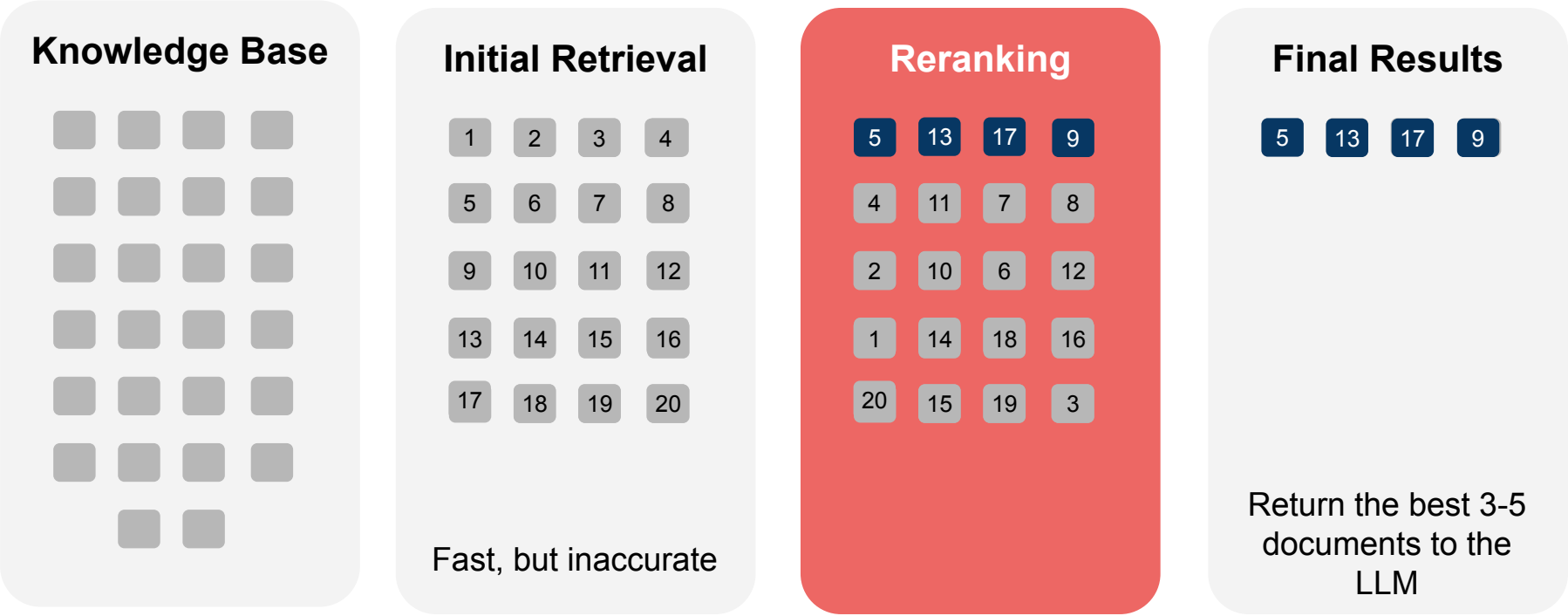
Contextual Retrieval Preprocessing



RAG Improvements

- RAG Introduction
- Keyword Search
- Semantic Search
- RAG Evaluations
- **RAG Improvements**
  - Query Rewriting
  - Contextual Retrieval
  - **Reranking**

Overview of Reranking



Original Query

What is the capital of Canada?

Semantically similar but **not** all directly **relevant**

Initial Results

Toronto is in Canada

The capital of France is in Paris

Canada is in the maple syrup capital of the world

Ottawa is the capital of Canada

Original Query

What is the capital of Canada?

You'll still return just 5-10 documents but reranking ensures they're far **more relevant.**

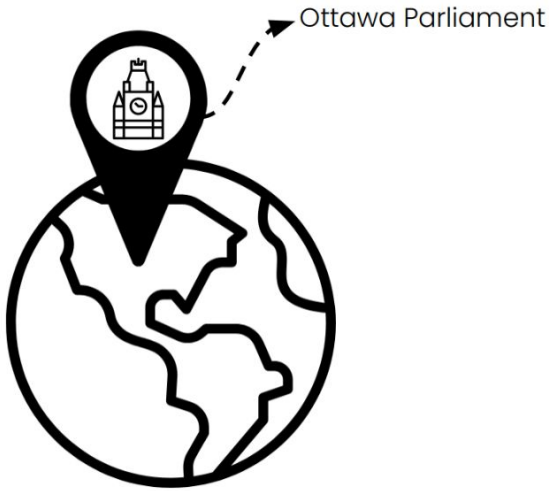
Reranked Results

Ottawa is the capital of Canada

Toronto is in Canada

The capital of France is in Paris

Canada is in the maple syrup capital of the world



Reranker

- Rerank models sort text inputs by semantic relevance to a specified query.
- Reranking models are **more accurate but more costly** than simple cosine similarity retriever
- Therefore, you do it after selecting top-k documents
- With reranker, you will retrieve more documents (higher k)
- Example of reranker: Cohere, Voyage
  - You can also use LLM to rerank - scoring relevance

Summary

- To allow LLMs to answer about specialized and novel knowledge, we need to **provide them with relevant context**.
- Therefore, we need to **retrieve relevant information**.
- Two information retrieval methods: **keyword search** and **semantic search**.
- To improve RAG, we can use **query rewriting** and **reranking**.